



Technical Documentation

Real Madrid C.F. ACWR Prediction and Planning Tool

Solution architecture, data model, modeling methodology, and interactive app guide

DOCUMENT PROFILE

Technical reference

Client context	Real Madrid C.F. fitness and performance staff
Project team	trAIIn Labs
Application	Streamlit ACWR monitor and 15-day workload forecast planner
Model stack	Three XGBoost daily-load regressors plus EWMA ACWR simulation
Version date	June 2026

Prepared as the technical reference for the deployed workload-planning prototype. ACWR outputs are workload-risk indicators and should be interpreted alongside coaching, medical, wellness, and match-context information.

Executive Summary

Purpose. This document describes the technical design of an interactive ACWR planning system for football fitness staff. The application converts coach-defined 15-day session plans into predicted external-load trajectories and then simulates Acute:Chronic Workload Ratio (ACWR) for every squad player across three load metrics.

Technical approach. The system uses a continuous player-day data model, three independent XGBoost regressors for daily load prediction, and deterministic EWMA equations for ACWR. This separation keeps the machine-learning task focused on session-to-load estimation while leaving workload-risk classification transparent and auditable.

Operational value. Coaches can plan the squad calendar, identify players forecast to enter high-risk zones, adjust an individual player’s plan, compare custom and squad forecasts, and export the final schedule in a PDF-ready layout.

28	15	3	6,310
squad players	forecast days	load metrics	player-days

Table 1: Executive view of the solution.

Dimension	Summary
Architecture	Streamlit application backed by cached model bundles and a processed daily player grid.
Data model	Period-level GPS/session records are cleaned, capped, aggregated, and expanded into a continuous player-day calendar.
Modeling	One XGBoost model per target load metric; predictions are clipped to non-negative daily loads.
ACWR method	7-day acute EWMA divided by 28-day chronic EWMA, with a 28-day warmup mask and explicit rest-day decay.
User workflow	Dashboard review, squad planning, forecast inspection, player-specific customization, and export.

Contents

Executive Summary	i
1 Project Scope	1
1.1 Core outputs	1
2 Solution Architecture	1
2.1 Runtime components	2
3 Data Model	2
3.1 Raw source	2
3.2 Session taxonomy	3
3.3 Processed daily table	3
3.4 Inference plan frame	4
4 Methodology	5
4.1 Cleaning and cohort definition	5
4.2 Feature engineering	5
4.3 Model training	5
4.4 Model and report figures	6
4.5 ACWR computation	7
4.6 Forecast algorithm	8
5 Interactive App User Guide	8
5.1 Dashboard	8
5.2 Planning and Forecast	8
5.3 Player Customization	11
5.4 Export Plan	12
6 Operations and Reproducibility	13
6.1 Common commands	13
6.2 Artifact contract	14
6.3 Testing and quality	14
7 Assumptions and Limitations	14
8 Summary	14

1 Project Scope

The application is an end-to-end decision-support tool for football workload planning. It does not predict injuries directly. Instead, it predicts future external training load from planned session types and converts those predicted loads into Acute:Chronic Workload Ratio (ACWR) trajectories.

The deployed application answers four operational questions:

1. What is the current ACWR status of the squad across the three monitored load metrics?
2. If coaches schedule a 15-day plan, which players are forecast to enter caution or danger zones?
3. Can an individual player's plan be modified to reduce forecast risk while preserving the squad plan?
4. Can the final squad plan be exported in a coach-readable PDF layout?

1.1 Core outputs

- Future daily load predictions for `total_distance`, `accelerations`, and `sprint_distance`.
- EWMA ACWR trajectories for each player and metric.
- Day-15 risk status and first date of danger-zone entry.
- A full-squad training schedule table with customized player plans marked.

2 Solution Architecture

The solution is split into three layers: a data/modeling package under `src/real_madrid_acwr`, a Streamlit application layer under `src/app`, and persisted artifacts under `data/processed` and `models/xgboost`.

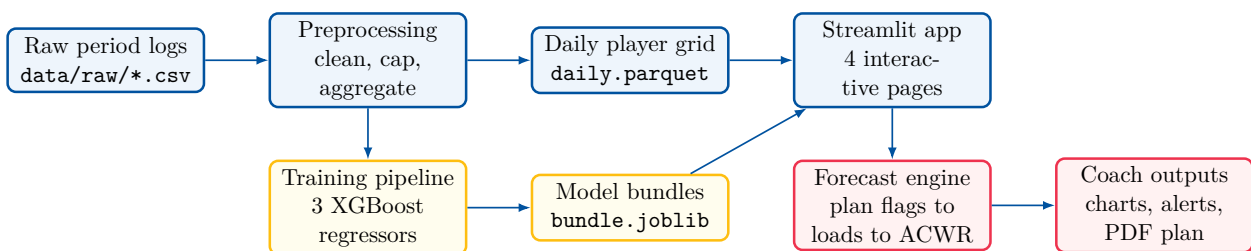


Figure 1: High-level system architecture. Production behavior is defined by the package under `src/`, the processed player-day table, and the model bundles.

2.1 Runtime components

Table 2: Runtime components and responsibilities. Component paths are shown relative to `src/`, except for `main.py`.

Component	Layer	Responsibility
<code>main.py</code> entry point	Streamlit shell	Dispatches the active page and injects app styling.
<code>app/pages.py</code> UI rendering	Application UI	Renders the Dashboard, Planning and Forecast, Player Customization, and Export Plan pages.
<code>app/planning.py</code> calendar state	Planning logic	Converts calendar events into the 15-day plan structure consumed by forecasting.
<code>app/forecasting.py</code> inference flow	Forecast engine	Builds inference frames, predicts future load, stitches forecast loads to history, and computes forecast ACWR.
<code>app/charts.py</code> visualization	Charting	Builds Plotly ACWR charts with historical and forecast load overlays.
<code>app/loaders.py</code> cached resources	Data access	Loads model bundles and <code>daily.parquet</code> ; computes current player ACWR in cached Streamlit resources.
<code>acwr.py</code> pure utility	Core package	Computes EWMA ACWR and classifies risk zones.
<code>datapipeline.py</code> preprocessing	Modeling package	Applies shared cleaning, aggregation, feature engineering, and daily-spine construction.
<code>training/*.py</code> model training	Modeling package	Trains the three target-specific XGBoost regressors and writes deployable artifacts.
<code>artifacts.py</code> serving contract	Artifact loading	Validates and loads model bundles before serving.

3 Data Model

3.1 Raw source

The bootstrap archive is `data/raw/data_acute_vs_chronic.zip`; when needed, the pipeline extracts it to `data/raw/data_acute_vs_chronic.csv`. The raw table contains one row per training period or drill block, not one row per player-day.

Table 3: Important raw fields and deployed interpretations.

Field	Interpretation
<code>player_id</code>	Player identifier used throughout the application.
<code>position_name_en</code>	Position label displayed in tables and player selectors.
<code>height</code> , <code>weight</code> , <code>date_of_birth</code>	Player profile features. Age is computed during cleaning and rounded to whole years by the current pipeline.
<code>period_start_time</code>	Period timestamp, normalized to a calendar date.
<code>period_name</code>	Session family prefix. Examples: G 1960, TAC 0133, BP 2351. Missing values are mapped to MATCH.
<code>activity_id</code>	Used to count period rows per player-day.
<code>total_distance</code>	Daily total-distance target after aggregation.
<code>acc_band7plus_total_effort_count</code>	Renamed to <code>accelerations</code> .
<code>velocity_band6plus7_total_distance</code>	Renamed to <code>sprint_distance</code> .

3.2 Session taxonomy

Table 4: Session families used in planning, modeling, and export.

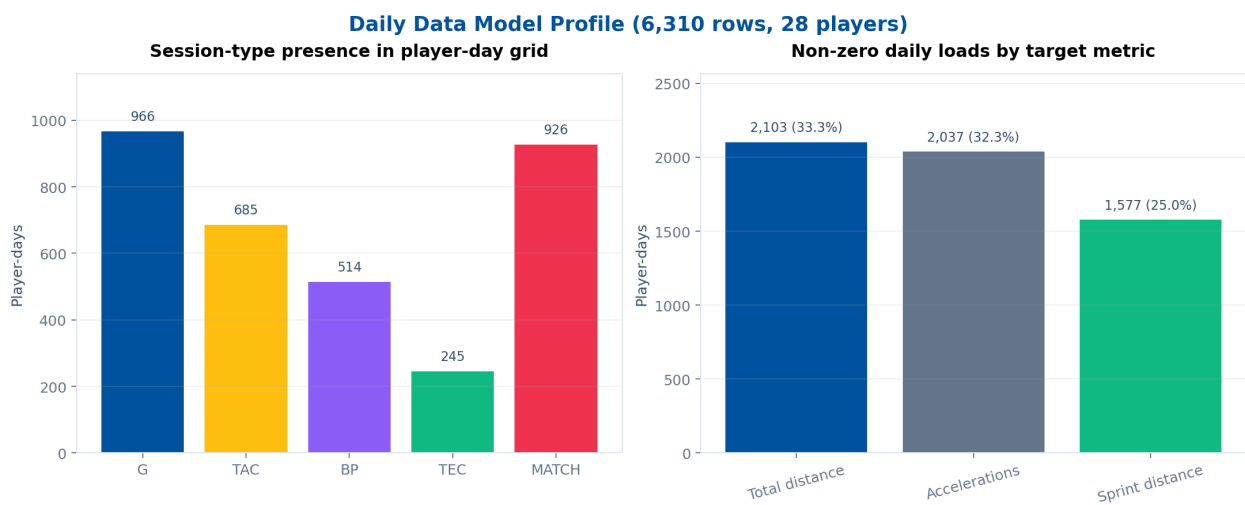
Code	Meaning
G	Game-based work / small-sided games.
TAC	Tactical training.
BP	Set pieces.
TEC	Technical training.
MATCH	Official match.
REST	No planned session. Encoded as zero load and zero session flags.

3.3 Processed daily table

The primary serving dataset is `data/processed/daily.parquet`. It contains a complete daily calendar spine for each retained player, with rest days inserted as explicit zero-load rows.

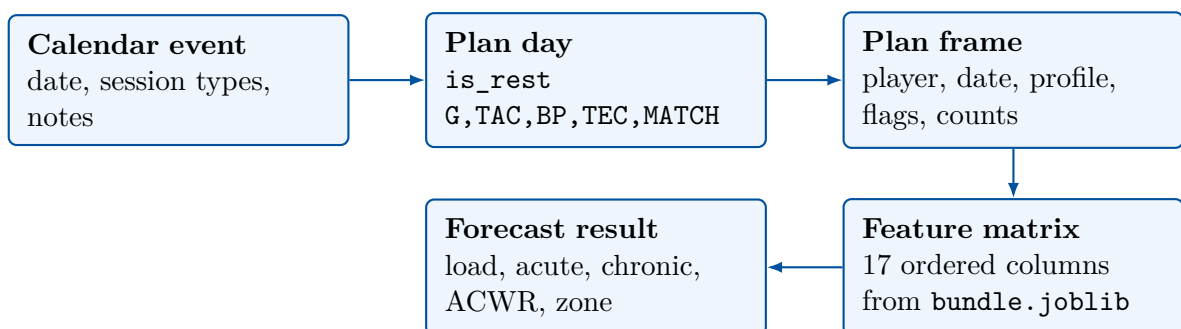
Table 5: Current processed data contract.

Property	Value
Rows	6,310 player-days
Players	28
Date range	2024-07-16 to 2025-06-26
Granularity	One row per player_id and calendar date
Load targets	total_distance, accelerations, sprint_distance
Session flags	has_BP, has_G, has_MATCH, has_TAC, has_TEC
Activity fields	n_periods, n_exercise_types
Profile fields	height, weight, age, position

**Figure 2:** Profile of the processed player-day grid. The left panel shows how often each session flag appears. The right panel shows the number and share of player-days with non-zero load by target metric.

3.4 Inference plan frame

At forecast time, the calendar plan is converted into one row per player and forecast day. For a 15-day horizon and 28 players, each target model receives 420 rows. Each row contains the planned session flags, activity counts, player profile fields, and day-of-week one-hot features.

**Figure 3:** Forecast data flow from calendar events to the final result object.

4 Methodology

4.1 Cleaning and cohort definition

The preprocessing pipeline applies the following steps:

1. Rename raw load columns to the deployed metric names.
2. Normalize timestamps and derive **exercise_type** from the session prefix.
3. Fill missing match flags, then drop the raw match flag after deriving session type.
4. Compute age from date of birth and session date; the current pipeline rounds age to whole years.
5. Drop rows with missing player metadata.
6. Apply IQR capping at $Q_3 + 3 \times IQR$ per player and exercise type for **total_distance**, and also for the target metric when training **accelerations** or **sprint_distance**.
7. Drop players with placeholder **weight == 200**.
8. Aggregate period rows into player-day rows.
9. Build a complete daily spine per player and fill rest days with zero load.

Rest-day filling is not cosmetic. ACWR is a daily time-series calculation, so missing calendar days must decay the acute and chronic EWMA states with zero load rather than disappear from the sequence.

4.2 Feature engineering

The deployed feature set is cross-sectional: all features are known before the planned day starts. The model does not receive lagged load, previous ACWR, player identifiers, or injury labels.

Table 6: Feature groups persisted in each deployed model bundle.

Group	Features
Activity volume	n_periods, n_exercise_types
Session family	has_BP, has_G, has_MATCH, has_TAC, has_TEC
Player profile	height, weight, age
Calendar	dow_0, dow_1, dow_2, dow_3, dow_4, dow_5, dow_6

The currently saved **bundle.joblib** artifacts each contain 17 ordered feature columns. The serving code always uses the feature order stored in the bundle, so training and inference stay aligned even if the column order in a DataFrame changes.

4.3 Model training

The system trains three independent XGBoost regressors: one for total distance, one for accelerations, and one for sprint distance. Each model is trained on a $\log(1 + y)$ -transformed target, with a **MinMaxScaler** fit on the training features only.

$$\tilde{y} = \log(1 + y) \quad (1)$$

$$\hat{y} = \max(\exp(\hat{\tilde{y}}) - 1, 0) \quad (2)$$

The training scripts use:

- random 80/20 train/test split with `random_state=42`;
- 10-fold shuffled KFold cross-validation;
- RandomizedSearchCV with 50 sampled configurations;
- `XGBRegressor(objective="reg:squarederror", tree_method="hist")`.

Table 7: Evaluation of the saved bundles on the deterministic held-out split.

Target	Unit	Features	RMSE	MAE	R^2
total_distance	metres	17	569.98	223.64	0.831
accelerations	count	17	2.73	1.09	0.750
sprint_distance	metres	17	15.02	5.38	0.352

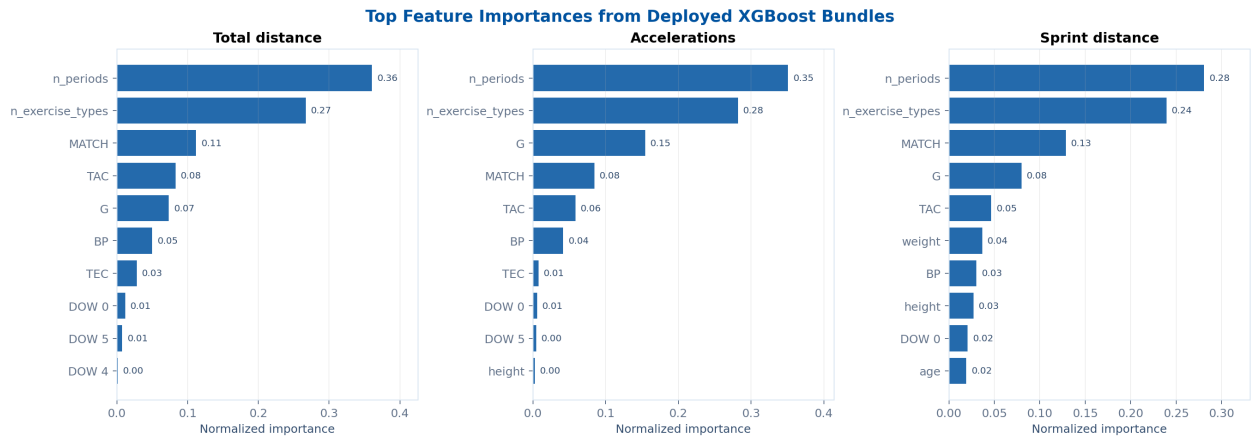


Figure 4: Top feature importances from the three deployed XGBoost bundles. Activity counts and session-family flags dominate the serving models.

4.4 Model and report figures

The repository also contains generated report figures under `reports/figures`. These are included here as model-development evidence and exploratory context.

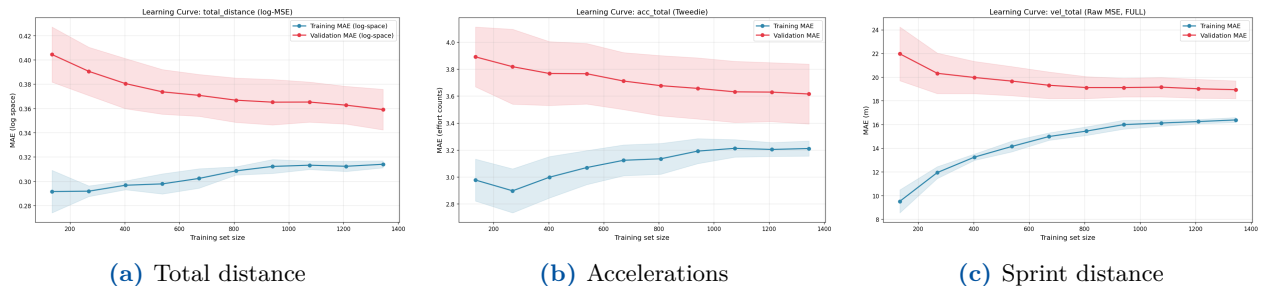


Figure 5: Learning-curve figures copied from the model reports.

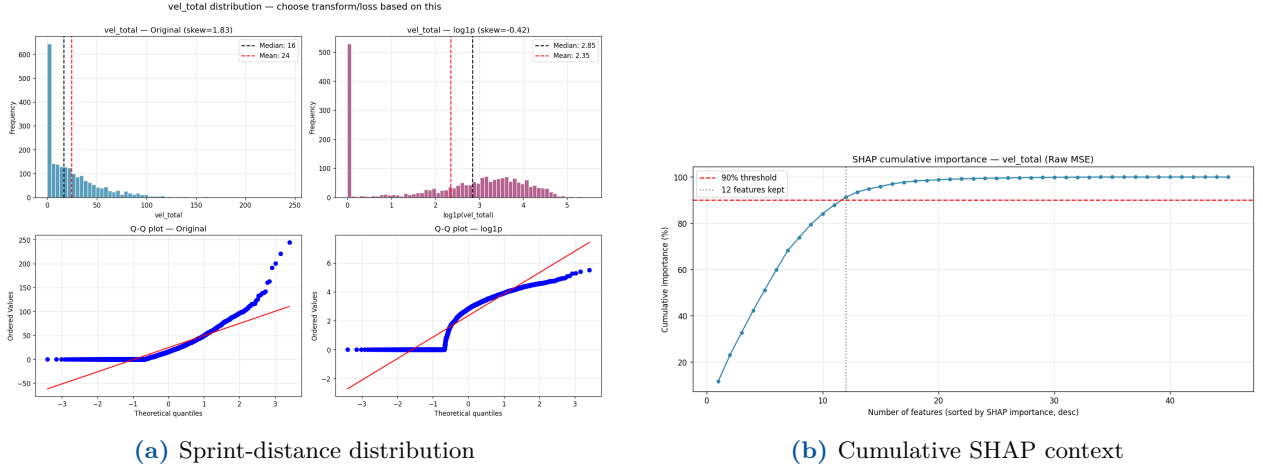


Figure 6: Additional report figures for the high-speed running target.

4.5 ACWR computation

ACWR is computed after load prediction. The models forecast load; the sport-science ratio is a deterministic transformation of historical and forecast daily load.

For a daily load series L_t , the system computes uncoupled exponentially weighted moving averages:

$$\text{acute}_t = \alpha_a L_t + (1 - \alpha_a) \text{acute}_{t-1}, \quad \alpha_a = \frac{2}{7 + 1} = 0.25 \quad (3)$$

$$\text{chronic}_t = \alpha_c L_t + (1 - \alpha_c) \text{chronic}_{t-1}, \quad \alpha_c = \frac{2}{28 + 1} \approx 0.069 \quad (4)$$

$$\text{ACWR}_t = \frac{\text{acute}_t}{\text{chronic}_t}$$

The first 28 rows of each player's history are masked because the chronic EWMA has not stabilized. If chronic load is zero, ACWR is reported as unavailable.

Table 8: Operational ACWR risk zones used by the application.

Zone	ACWR range	Interpretation
Under	< 0.8	Insufficient load stimulus or recent unloading.
Optimal	$0.8 \leq \text{ACWR} < 1.3$	Target training range.
Caution	$1.3 \leq \text{ACWR} < 1.5$	Elevated workload signal; monitor closely.
Danger	≥ 1.5	High workload spike signal; consider reducing load.

4.6 Forecast algorithm

The forecast engine is direct and non-recursive:

1. Convert calendar events into 15 plan-day dictionaries.
2. Build a plan frame with one row per player and day.
3. Add day-of-week one-hot columns.
4. Align the matrix to `feature_cols` from each `bundle.joblib`.
5. Predict all plan rows for a target in one XGBoost call.
6. Force rest-day predictions to zero where `n_periods == 0`.
7. Append the 15 predicted loads to the player's historical daily loads.
8. Recompute EWMA ACWR on the stitched history-plus-forecast series.
9. Extract forecast curves, day-15 ACWR, and risk zones.

This separation keeps the model interpretable: the regressor estimates daily external load from the planned session composition, while the ACWR module simulates how that load path changes short-term versus long-term workload state.

5 Interactive App User Guide

Run the app from the project root:

```
python -m pip install -e ".[dev]"
streamlit run main.py
```

The sidebar contains the page navigation and ENG/ESP language toggle. The application has four pages.

5.1 Dashboard

The Dashboard summarizes the current state of the squad using the latest available historical data. It explains the ACWR formula, shows the EWMA smoothing setup, defines risk zones, and lists each player with current ACWR values for the three load metrics.

Recommended use:

1. Review high-risk and caution counts before planning the next cycle.
2. Check whether risk is isolated to one metric or appears across multiple metrics.
3. Treat the values as workload signals, not medical diagnoses.

5.2 Planning and Forecast

The Planning and Forecast page is the main workflow for squad planning. Coaches create calendar events inside the 15-day forecast window, assign one or more session types, and run the forecast.

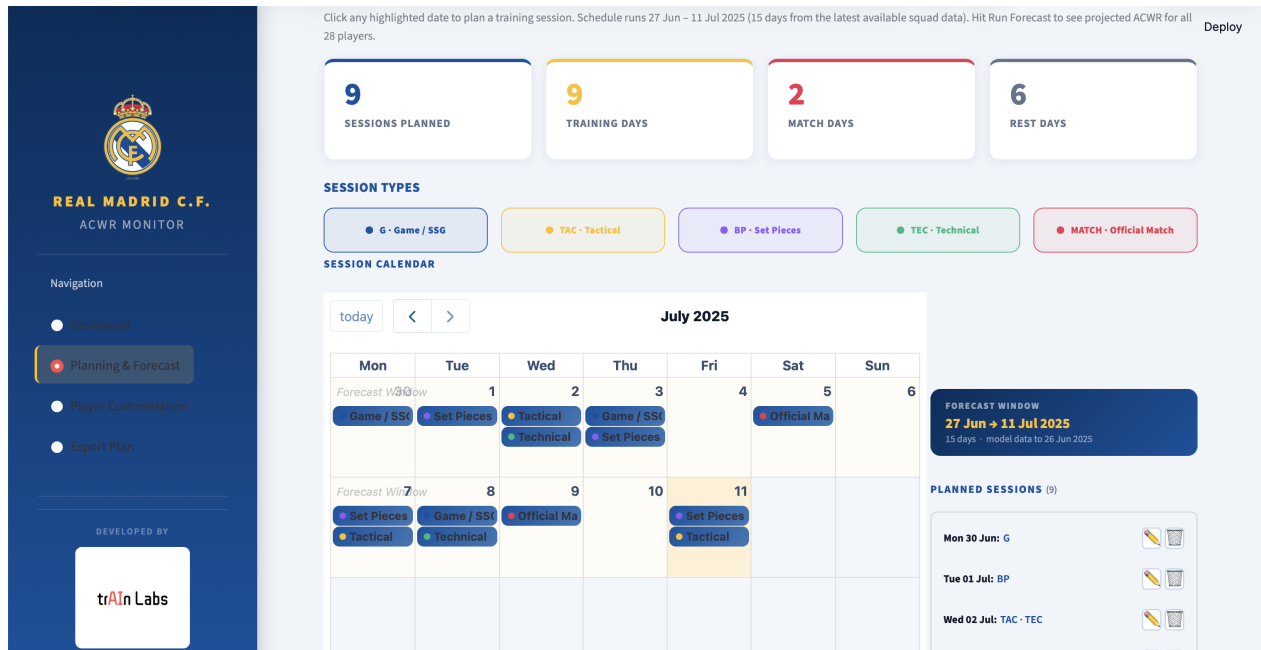


Figure 7: Planning page with FullCalendar event creation, plan KPIs, session-type legend, and the planned-session list.

Workflow:

1. Click a highlighted date in the forecast window.
2. Select session types: G, TAC, BP, TEC, or MATCH.
3. Save the event. Multiple session types can be scheduled on the same day.
4. Repeat until the 15-day plan is complete.
5. Click **Run Forecast**.

After forecasting, the app shows an injury-risk alert if any player enters the danger zone during the forecast horizon.

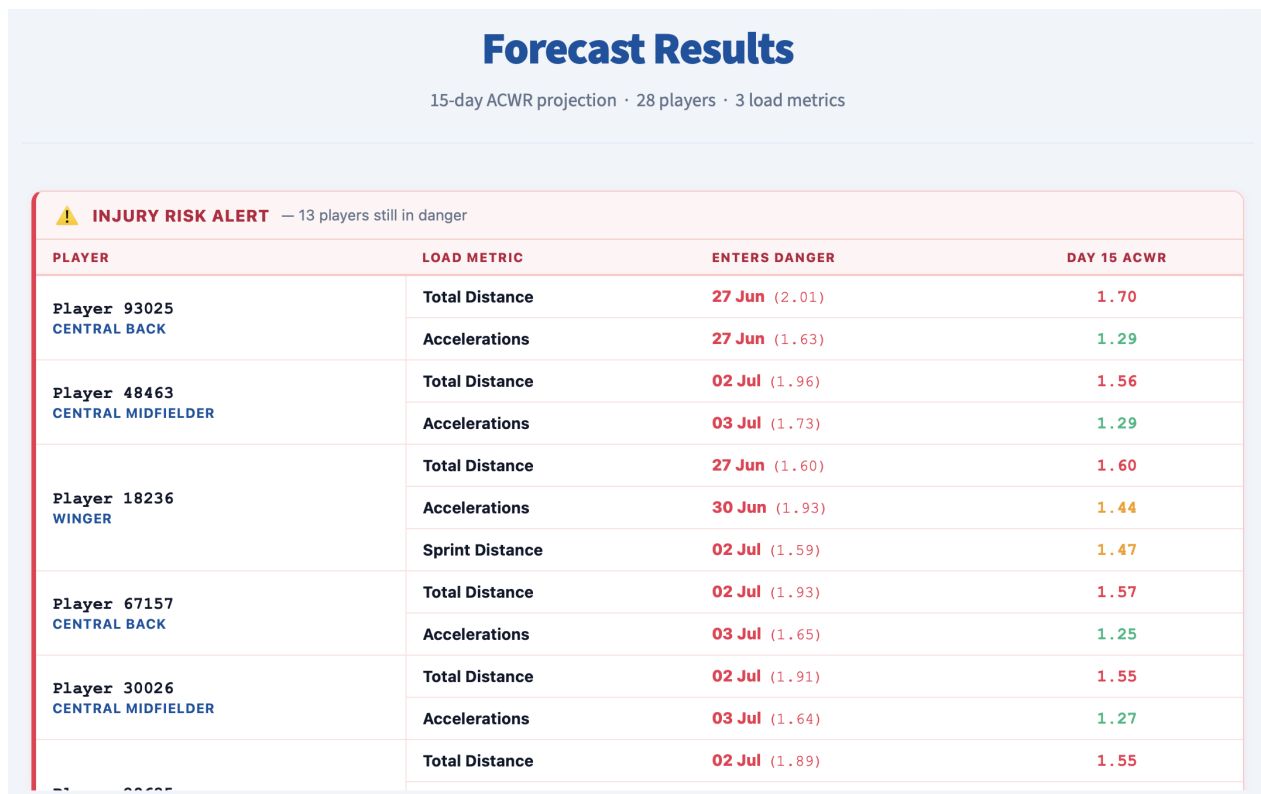


Figure 8: Forecast alert table listing players and metrics that enter the danger zone.

For a selected player, the page renders one chart per metric. The chart can optionally show load bars beneath the ACWR line.

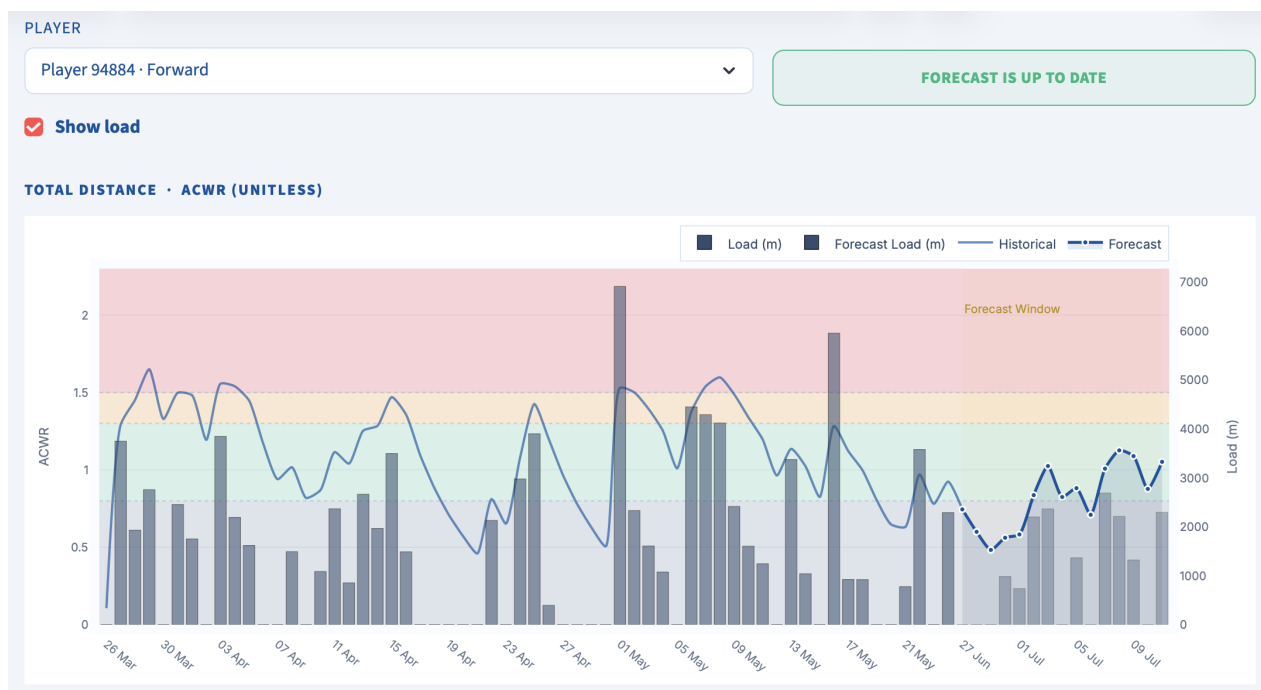


Figure 9: Per-player forecast chart with historical ACWR, forecast ACWR, load bars, risk bands, and the forecast-window overlay.

The Day-15 summary table gives the final forecast ACWR for all players and all metrics. It is the fastest way to compare who remains in danger, who is merely in caution, and who ends the horizon

in an optimal or undertraining zone.

DAY-15 SUMMARY — ALL PLAYERS					
PLAYER	POSITION	TOTAL DISTANCE	ACCELERATIONS	SPRINT DISTANCE	STATUS
Player 11983	CENTRAL MIDFIELDER	1.56 DANGER	1.30 CAUTION	1.02 OPTIMAL	HIGH RISK
Player 16464	CENTRAL BACK	— —	— —	— —	OK
Player 18236	WINGER	1.60 DANGER	1.44 CAUTION	1.47 CAUTION	HIGH RISK
Player 21190	FULL BACK	1.57 DANGER	1.28 OPTIMAL	1.20 OPTIMAL	HIGH RISK
Player 21206	CENTRAL MIDFIELDER	1.10 OPTIMAL	0.75 UNDER	0.48 UNDER	LOW
Player 28635	WINGER	1.55 DANGER	1.30 CAUTION	1.16 OPTIMAL	HIGH RISK
Player 30026	CENTRAL MIDFIELDER	1.55 DANGER	1.27 OPTIMAL	1.06 OPTIMAL	HIGH RISK
Player 31083	CENTRAL BACK	1.61 DANGER	1.20 OPTIMAL	0.84 OPTIMAL	HIGH RISK
Player 37191	CENTRAL BACK	1.12 OPTIMAL	0.74 UNDER	0.56 UNDER	LOW
Player 39471	WINGER	1.53 DANGER	1.34 CAUTION	1.28 OPTIMAL	HIGH RISK
Player 43211	CENTRAL BACK	— —	— —	— —	OK

Figure 10: Day-15 summary table across all players and all monitored load metrics.

5.3 Player Customization

The Player Customization page starts from the squad plan and lets coaches modify it for a single player. It is designed for players flagged as at risk after a squad forecast.

Player Customization

Modify the squad plan for an individual player and compare the custom ACWR forecast against the original.

AT RISK

Player 11983 Player 18236 Player 21190 Player 28635 Player 30026 Player 31083 Player 39471 Player 48463 Player 55220 Player 59105

Player 59915 Player 62110 Player 64878 Player 67157 Player 70334 Player 93025 Player 93988

☐ Show all players (not just at-risk)

SELECT PLAYER TO CUSTOMISE

Player 11983 · Central Midfielder

Player 11983

CENTRAL MIDFIELDER

Last active: 26 Jun 2025

Active days: 333

Enters danger: 02 Jul

SQUAD PLAN — DAY 15 FORECAST

TOTAL DISTANCE 1.56 DANGER	ACCELERATIONS 1.30 CAUTION	SPRINT DISTANCE 1.02 OPTIMAL
--	---	--

CUSTOM PLAN — MODIFY FROM SQUAD PLAN

Each day is pre-loaded from the squad plan. Adjust rest days or session types for this player only.

Day 1 Fri 27 Jun	☒ Rest
Rest	
Day 2 Sat 28 Jun	☒ Rest
Rest	

Figure 11: Player-specific customization page with at-risk chips, selected player profile, squad forecast status, and editable 15-day plan rows.

Workflow:

1. Run a squad forecast first on the Planning and Forecast page.
2. Open Player Customization.
3. Select an at-risk player, or enable the option to show all players.
4. Modify rest days and session types for that player only.
5. Run the player forecast.
6. Compare squad-plan ACWR against custom-plan ACWR for each target.

The custom plan is stored in session state and is used by the Export Plan page. Players with customized plans are marked with a custom badge in the export table.

5.4 Export Plan

The Export Plan page converts the effective squad plan into an A3 landscape, browser-printable HTML schedule. It uses the squad plan for most players and the customized plan for any player modified on the Player Customization page.

Export Plan

Full squad training plan. Click Export PDF to save or print.

↓ Export PDF



Training Plan

Real Madrid C.F. · Season 2024/25 · 15-Day Schedule · 28 Players

1 player has a customised plan (marked **CUSTOM**).

PLAYER	FRI 27 JUN	SAT 28 JUN	SUN 29 JUN	MON 30 JUN	TUE 01 JUL	WED 02 JUL	THU 03 JUL	FRI 04 JUL	SAT 05 JUL	SUN 06 JUL	MON 07 JUL
21190 FULL BACK	REST	REST	REST	G	BP	TAC TEC	G BP	REST	MATCH	REST	1
55220 FULL BACK	REST	REST	REST	G	BP	TAC TEC	G BP	REST	MATCH	REST	1
64878 FULL BACK	REST	REST	REST	G	BP	TAC TEC	G BP	REST	MATCH	REST	1
70334 FULL BACK	REST	REST	REST	G	BP	TAC TEC	G BP	REST	MATCH	REST	1
84160 FULL BACK	REST	REST	REST	G	BP	TAC TEC	G BP	REST	MATCH	REST	1
88732 FULL BACK	REST	REST	REST	G	BP	TAC TEC	G BP	REST	MATCH	REST	1
16464 CENTRAL BACK	REST	REST	REST	G	BP	TAC TEC	G BP	REST	MATCH	REST	1

Figure 12: Export page showing the full training schedule table and the export-PDF control.

Workflow:

1. Run a squad forecast so the plan exists in app state.
2. Optionally customize selected players.
3. Open Export Plan.
4. Review the full table by player and date.
5. Click **Export PDF**; the browser print dialog saves or prints the plan.

6 Operations and Reproducibility

6.1 Common commands

```
python -m pip install -e ".[dev]"
python train_models.py
streamlit run main.py
make quality
```

`python train_models.py` performs the production build: raw extraction, cleaning, outlier treatment, daily-grid creation, feature engineering, model training, and artifact writing.

6.2 Artifact contract

Each deployed target has exactly one artifact:

```
models/xgboost/{target}/bundle.joblib
```

Each bundle must contain:

- `model`: fitted `XGBRegressor`;
- `scaler`: fitted `MinMaxScaler`;
- `feature_cols`: ordered list of feature names;
- `ewma_spans`: `"acute"`: 7 and `"chronic"`: 28.

The app validates all bundles before rendering. If a bundle is missing or malformed, the app stops and asks the operator to regenerate artifacts with `python train_models.py`.

6.3 Testing and quality

The repository includes tests for ACWR utilities, planning helpers, forecast behavior, page contracts, and artifact loading. The standard quality command is:

```
make quality
```

This runs Ruff, mypy, and pytest.

7 Assumptions and Limitations

- The data covers one season only, from July 2024 through June 2025.
- The model does not include wellness, session duration, RPE, medical status, or tactical role context.
- Rest days and data-capture gaps are indistinguishable once the daily spine is filled.
- ACWR is a workload proxy and should not be read as an injury probability.
- The risk thresholds 0.8, 1.3, and 1.5 are operational sport-science heuristics.
- The feature set is cross-sectional; future predictions do not feed into later-day feature vectors. History affects the final ACWR through the EWMA stitching step, not through the load model.
- Sprint distance remains the noisiest target and has the weakest held-out R^2 , so coaches should interpret that metric with proportionally more caution.

8 Summary

The project implements a two-stage workload planning system. First, XGBoost models predict daily external load from player profile and planned session composition. Second, deterministic EWMA ACWR utilities stitch predicted loads onto each player's historical daily series and classify the resulting ratio into operational risk zones. The Streamlit interface turns that pipeline into a coaching workflow: inspect current risk, plan a 15-day schedule, review forecast alerts and charts, customize plans for at-risk players, and export the final training plan.